C++ Classes
&
Object Oriented Programming

Structures and Classes in C++

- C++ supports all the features of structures as defined in C.
- In C++, a structure can have both variables and functions as members.
- It can declare some of its members as ‘private’.
- In C++, the structure names are stand-alone and can be used like any other type names.
- **student A; // C++ declaration**
- By default the members of a class are private, while, by default, the members of a structure are public.

Object Oriented Programming

- Object-oriented programming (OOP)
  - Encapsulates data (attributes) and functions (behavior) into packages called classes.
- So, Classes are user-defined (programmer-defined) types.
  - Data (data members)
  - Functions (member functions or methods)
- In other words, they are structures + functions

Classes in C++

- A class definition begins with the keyword *class*.
- The body of the class is contained within a set of braces, { } ; (notice the semi-colon).
- Class class_name

```
{
        .............
         ..............
         .............
};
```

Classes in C++

- Within the body, the keywords *private:* and *public:* specify the access level of the members of the class.
  - The default is private.

- Usually, the data members of a class are declared in the *private:* section of the class and the member functions are in *public:* section.

- Class class_name

```
{
        private:        .............
                        ...............
                          .............
         public:        ...............
                          .............
};
```

Classes in C++

- <u>Member access specifiers</u>
  - public:
    - can be accessed outside the class directly.
      - The public stuff is *the interface*.
  - private:
    - Accessible only to member functions of class
    - Private members and methods are for internal use only.

- This class example shows how we can encapsulate (gather) a circle information into one package (unit or class)

```
class Circle
{
    private:
   double radius;
    public:
   void setRadius(double r);
      double  getDiameter();
   double getArea();
   double getCircumference();
};
```

Creating an object of a Class

- Declaring a variable of a class type creates an object. You can have many variables of the same type (class).
  - *Instantiation*
- Once an object of a certain class is instantiated, a new memory location is created for it to store its data members and code
- You can instantiate many objects from a class type.
  - Ex) Circle c; Circle *c;

Special Member Functions

- Constructor:
  - Public function member
  - called when a new object is created (instantiated).
  - Initialize data members.
  - Same name as class
  - No return type
  - Several constructors
  - Function overloading

Special Member Functions

```
class Circle
{
    private:
   double radius;
    public:
   Circle();
   Circle(int r);
    void setRadius(double r);
   double getDiameter();
   double getArea();
   double getCircumference();
};
```

Special Member Functions

- Class implementation: writing the code of class methods.

- There are two ways:
  - Member functions defined outside class
    - Using Binary scope resolution operator (`::`)
    - “Ties” member name to class name
    - Uniquely identify functions of particular class
    - Different classes can have member functions with same name
  - Format for defining member functions

```
ReturnType ClassName::MemberFunctionName()
{
…
}
```

- Member functions defined inside class
  - Do not need scope resolution operator, class name

```
class Circle
{
    private:
   double radius;
    public:
   Circle() { radius = 0.0;}
   Circle(int r);
   void setRadius(double r){radius = r;}
   double getDiameter(){ return radius *2;}
   double getArea();
   double getCircumference();
};
```

```
class Circle
{
    private:
double radius;
    public:
Circle() { radius = 0.0;}
Circle(int r);
void setRadius(double r){radius = r;}
double getDiameter(){ return radius *2;}
double getArea();
double getCircumference();
};
Circle::Circle(int r)
{
    radius = r;
}
double Circle::getArea()
{
    return radius * radius * (22.0/7);
}
double Circle:: getCircumference()
{
    return 2 * radius  * (22.0/7);
}
```

Defined outside class

Accessing Class Members

- Operators to access class members
  - Identical to those for **`struct`**s
  - Dot member selection operator (`.`)
    - Object
    - Reference to object
  - Arrow member selection operator (`->`)
    - Pointers

```
class Circle
{
    private:
double radius;
    public:
Circle() { radius = 0.0;}
Circle(int r);
void setRadius(double r){radius = r;}
double getDiameter(){ return radius *2;}
double getArea();
double getCircumference();
};
Circle::Circle(int r)
{
    radius = r;
}
double Circle::getArea()
{
    return radius * radius * (22.0/7);
}
double Circle:: getCircumference()
{
    return 2 * radius  * (22.0/7);
}
```

```
void main()
{
    Circle c1,c2(7);

    cout<<"The area of c1:"
        <<c1.getArea()<<"\n";

    //c1.raduis = 5;//syntax error
    c1.setRadius(5);

    cout<<"The circumference of c1:"
<< c1.getCircumference()<<"\n";

    cout<<"The Diameter of c2:"
<<c2.getDiameter()<<"\n";
}
```

```
class Circle
{
    private:
double radius;
    public:
Circle() { radius = 0.0;}
Circle(int r);
void setRadius(double r){radius = r;}
double getDiameter(){ return radius *2;}
double getArea();
double getCircumference();
};
Circle::Circle(int r)
{
    radius = r;
}
double Circle::getArea()
{
    return radius * radius * (22.0/7);
}
double Circle:: getCircumference()
{
    return 2 * radius  * (22.0/7);
}
```

```
void main()
{
    Circle c(7);
    Circle *cp1 = &c;
    Circle *cp2 = new Circle(7);

    cout<<"The are of cp2:"
    <<cp2->getArea();

}
```

Memory Allocation for Objects

- The member functions are created and placed in the memory space only once when they are defined.
- Since all the objects belongs to that class use the same member functions, no separate space is allocated for member functions when the objects are created.
- Only space for member variables is allocated separately for each object.
- Separate memory locations for the objects are essential, because the member variables hold different data values for different objects.

Using Default Arguments

- When you don't provide enough arguments in a function call, you usually want the compiler to issue a warning message for this error
- Sometimes it is useful to create a function that supplies a default value for any missing parameters

```
#include<iostream.h>
void main(void)
  {
     // function prototype – this function requires 3 arguments
     int compute(int length, int width, int height);
     // function call – one parameter is missing
     cout<<compute(12, 7);
  }
```

Figure 4-37 Incorrect call to a function

Using Default Arguments

```
#include<iostream.h>
void main()
  {
    double compute(int length, int width, int height = 1);
    cout<<compute(4, 4, 4); // uses three values – result is 64
    cout<<calculate(4, 4);
    //  uses two values and 1 – //result is 16
  }
int calculate(int length, int width, int height)
   {
    return(length * width * height);
  }
```

Figure 4-38 Program using a function with a default parameter

-
  - **If you assign a default value to any variable in a function prototype's parameter list, then all parameters to the right of that variable also must have default values**
  - **If you omit any argument when you call a function that has default parameters, then you also must leave out all arguments to the right of that argument**

Examples of Legal and Illegal Use of Functions with Default Parameters

Ex3-16.cpp

```
#include<iostream.h>
void main()
  {
    void badFunc(int var1 = 1, int var2);
          // illegal – if var1 has a default, then var2 must
    void functionWithDefaults(int var1,int var2=2,int var3=3);
          // legal
    functionWithDefaults();
        // illegal – a value for var1 is mandatory
    functionWithDefaults(4); // legal – output is 423
    functionWithDefaults(4, 5);  // legal – output is 453
    functionWithDefaults(4, 5, 6); // legal – output is 456
  }
    void functionWithDefaults(int one, int two, int three)
  {
    cout<<one<<two<<three;
  }
```

Figure 4-39 Examples of legal and illegal use of functions with default parameters

Arrays of Objects

- Arrays of variables that are of type class are called ***arrays of objects***.
- Array of objects behave like any other

manager [i]. putdata( );
to execute the putdata( )
member function of the i
element of the array manager.

```
class employee
{
        char name [30];
        float age;
    public:
        void getdata (void);
        void putdata (void);
};

employee manager [5];
employee worker [25];
```

•
•
•

Objects as Function Arguments

- An object can be used as a function argument like any other data type.
- Two ways:
- A copy of the entire object is passed to the function. ***(Pass-by-Value)***
- Only the address of the object is transferred to the function. ***(Pass-by-Reference)***
- The pass-by-reference method is more efficient since it requires to pass only the address of the object and not the entire object.

Objects as Function Arguments

- An object can also be passed as an argument to a non-member function.
- Such functions can have access to the public member functions only through the objects passed as arguments to it.
- These functions cannot have access to the private data members.

Friendly Functions

- The private members can not be accessed from outside the class.

- A non-member function can not have an access to the private data of a class.

- C++ allows a common function to be made friendly with more than one classes, thereby allowing the function to have access to the private data of these classes.

- Such a function need not be a member of these classes.
- To make an outside function friendly to a class, we have to simply declare this function as a friend of the class.

```
#include <iostream>
Using namespace std;
class exforsys {
Private: int a,b;
Public: void test(){
a=100;
b=200;}
friend int compute(exforsys e1)
};
int compute(exforsys e1){
//Friend Function Definition which has access to private
data
return (e.a+e.b)-5;}
main(){
exforsys e;
e.test();
cout<<"The result is:"<< compute( e);
//Calling of Friend Function with object as argument.
}
```

Friendly Functions

- Special Characteristics:
- It is not in the scope of the class to which it has been declared as friend.

- Since it is not in the scope of the class, it cannot be called using the object of the class.

- It can be invoked like a normal function without the help of any object.
- It is possible to declare the friend function as either private or public

Friendly Functions

- Unlike member functions, it cannot access the member names directly and has to use an object name and dot membership operator with each member name.

- It can be declared either in the public or private part of a class without affecting its meaning.

- Usually, it has objects as arguments.
- Member function of one class can be friend functions of another class.

- In such cases, they are defined using the scope resolution operator as:

Friendly Functions

```
class X
{
    …
    …
    int fun1 ( );
    …
};
```

```
class Y
{
    …
    …
    friend int X  : :  fun1 (
);
    …
};
```

We can also declare all the member functions of one class as the friend functions of another class.

In such cases, the class is called a ***friend class***.

```
class Z
{
        …
        …
        friend class X ;
        …
};
```

Inline Functions

- Each time you call a function in a C++ program, the computer must do the following:
  - Remember where to return when the function eventually ends
  - Provide memory for the function's variables
  - Provide memory for any value returned by the function
  - Pass control to the function
  - Pass control back to the calling program
- This extra activity constitutes the **overhead**, or cost of doing business, involved in calling a function

Using an Inline Function

```
#include<iostream.h>
// note the keyword inline in the function header
inline double computeGross(double hours, double rate)
  {
      return(hours * rate);
  }
void main()
  {
    double hrsWorked=37.5, rateOfPay=12.45, gross;
    gross = computeGross(hrsWorked, rateOfPay);
          // the call to the inline function
          // looks like a regular function call
    cout<<endl<<"nGross pay is "<<gross;
  }
```

Figure 4-36 Using an inline function

Inline Functions

- An inline function is a small function with no calling overhead
- Overhead is avoided because program control never transfers to the function
- A copy of the function statements is placed directly into the compiled calling program
- The inline function appears prior to the main(), which calls it
- Any inline function must precede any function that calls it, which eliminates the need for prototyping in the calling function

Inline functions

Some situations where inline expansion may not work are:

1)For functions returning values,if a loop,a switch,or a goto exists.

2)For a function not returning values ,if a return statement exists.

3)If functions contain static variables

4)If inline functions are recursive

Static Data Members

- A data member of a class can be qualified as static.
- **Characteristics of static member variables:**
- It is initialized to zero when the first object of its class is created. No other initialization is permitted.

- Only one copy of that member is created for the entire class and is shared by all the objects of that class, no matter how many objects are created.

- It is visible only within the class, but its lifetime is the entire program.
- Static variables are normally used to maintain values common to the entire class.

Static Data Members

- The type and scope of each static member variable must be defined outside the class definition.
- This is because the static data members are stored separately rather than as a part of an object.
- Since they are associated with class itself rather than with any class object, they are also known as class variables.

Static Data Members

- Static variables are declared in class and defined outside the class.

- While defining a static variable, some initial value can also be assigned to the variable.

- type class-name : : static-variable = initial value;

- Like static member variable, we can also have static member functions.

- **Properties of member functions:**
- A static function can have access to only other static members (functions or variables).

- A static member function can be called using the class name (instead of its objects) as:
- ***class-name : : function-name;***

Type cast operator

- Converting an expression of a given type into another type is known as type-casting.
- **Syntax: (type name)expression *//c* notation**
- **Type name (expression) *//c++***

  notation
- **EX:short a=2000;**
- **int b;**
- **b = (int) a; *// c-like cast notation***
- **b = int (a); *// c++ notation or function call notation***

Type cast operator

- If the type is an identifier then c++ notation holds good .
- **EX:**

  P = int * (q); // illegal

  P = (int *) q; // legal
- Another alternative is use typedef to create identifier of required type.
- **EX:**
- **typedef int * int_ptr;**
- **P = int_ptr(q);**

function overloading

- Function overloading is one of the most powerful features of C++ programming language.
- A function is said to be overloaded when same name is given to different functions.
- The number of parameters, the data type of parameters, and the order of appearance these three together are referred to as the function signature.
- While overloading a function, the return types of the function need not differ

example

```
#include<conio>
 #include<iostream>
 class arith
{ public:
  void calc(int num1)
   { count<<"\n\nSquare of a given number: "
            <<num1*num1 <<endl; }
  void calc(int num1, int num2 ) { cout<<"product
   of a given numbers: "    <<num1*num2 <<endl; }
};
void main()
 { clrscr(); arith a;
   a.calc(5);
   a.calc(6,7); getch(); }
```

Function overloading

- A function call first matches the prototype having same number and type of argument and then calls the appropriate function for execution.
- The function selection involves the following steps:
- 1) The compiler first tries to find an exact match in which the types of actual arguments are the same,and use that function.
- 2) If exact match is not found ,the compiler uses the integral promotions to the actual arguments to find a match.

Function overloading

3)when either of them fails,the compiler tries to use the built-in conversions to actual arguments and then uses the function whose match is unique.If the conversion leads to multiple choices ,then the compiler will generate error message.

EX: long square(long n);
long square(double x);
Function call : square(10);

4)If all of the steps fail,then the compiler will try the user-defined conversions in combination with integral promotion and built-in conversion to find a unique match.

Function overloading

```
# include<iostream>
using namespace std;
int area(int side)
{
   return side*side;
}
int area(int l , int b)
{
   return l*b;
}
Int main()
{ cout << area(10)<<endl;
  Cout << area( 2,3)<<endl; return 0;}
```

Three Overloaded Functions That Perform Similar Tasks

Ex3-17.cpp
Ex3-18.cpp

```
#include<iostream.h>
void squareValue(int x)
  {
    cout<<"In integer function "<<x*x<<endl;
  }
void squareValue(float x)
  {
    cout<<"In float function "<<x*x<<endl;
  }
void squareValue(double x)
  {
    cout<<"In double function "<<x*x<<endl;
  }
void main()
  {
    int i = 5;
    float f = 2.2;
    double d = 3.3;
    squareValue(i); // output is "In integer function 25"
    squareValue(f); // output is "In float function 4.84"
    squareValue(d); // output is "In double function 10.89"
  }
```

Figure 4-41 Three overloaded functions that perform similar tasks

Returning Objects

Like a function can receive objects as arguments, it can also return objects.

Const Member Functions

- If a member function does not alter any data in the class, then it is called a const member function.
- void mul (int, int) const ;
- void get_balance( ) const ;

- The qualifier const is appended to the function prototypes (in both declaration and definition). The compiler will generate an error message if such functions try to alter the data values.

Pointer To Members

- It is possible to take the address of a member of a class and assign it to a pointer.
- The address of a member can be obtained by applying the operator & to a fully qualified class member name.
- A class member pointer can be declared using the operator : : * with the class name.

Pointer To Members

We can define a pointer to the member m as follows:

int A : : * pm = &A : : m;

A : : * "pointer-to-member of A class".

&A : : m means "address of the m member of A class".

```
class A
{
   private :
      int m ;
   public :
      void show( ) ;
} ;
```

Pointer To Members

The dereferencing operator .* is used when the object itself is used with the member pointer.

The dereferencing operator ->* is used to access a member when we use pointers to both the object and the member.

```
class A
{
   private :
      int m ;
   public :
      void show( ) ;
} ;
A a ;
int A  : : * pm = & A : : m ;
A * pa = & a ;
```

Pointer To Members

```
class A
{
   private :
      int m ;
   public :
      void show( ) ;} ;
A a ;
int A  : : * pm = & A : : m ;
A * pa = & a ;
```

To refer the member m
a .* pm

To refer the member m
pa -> * pm

•We can also design pointers to member functions which, then, can be invoked using the dereferencing operators in the main.

(object-name . * pointer-to-member function) ( )

(po
function) ( )

The precedence of ( ) is higher than that of . * and -> * , so the parentheses are necessary.

```
#include <iostream>
using namespace std;
class X { public:  int a;
                                void f(int b) {
    cout << "The value of b is "<< b << endl; }};
int main() {
  // declare pointer to data member
  int X::*ptiptr = &X::a;
  // declare a pointer to member function
  void (X::* ptfptr) (int) = &X::f;
  // create an object of class type X
  X xobject;
  // initialize data member
  xobject.*ptiptr = 10;
  cout << "The value of a is " << xobject.*ptiptr <<
endl;
  // call member function
  (xobject.*ptfptr) (20);
}
```

Local Classes

- Classes can be defined and used inside a function or a block. Such classes are called local classes.
- Local classes can be used global variables and static variables but can not use automatic variables. The global variables should be used with the scope operator (: :).
- They cannot have static data members and member functions must be defined inside the local classes.

```
int x;                          // global variable
void f()                        // function definition
{   static int y;     // static variable y can be
                                          used by local class
     int x;         // auto variable x cannot be used by
                              // local class
     extern int g();     // extern function g can be
                              used by local class
  class local {                           // local class
   int g() { return x; }       // error, local variable x
                              // cannot be used by g
    int h() { return y; }      // valid,static variable y
     int k() { return ::x; }      // valid, global x
     int l() { return g(); }    // valid, extern function g
     };}
int main()
{      local* z;   // error: the class local is not visible
     }
```

A local class cannot have static data members. In the following example, an attempt to define a static member of a local class causes an error:

```
void f()
{
   class local
   {
      int f();              // error, local class has noninline
                            // member function
      int g() {return 0;}   // valid, inline member function
      static int a;         // error, static is not allowed for
                            // local class
      int b;                // valid, nonstatic variable
   };}
```

An enclosing function has no special access to members of the local class.